



INSTITUTO FEDERAL DE MINAS GERAIS

Bacharelado em Ciência da Computação

Disciplina: Pesquisa Operacional

Trabalho Prático - Partes 2 e 3

Professor: Diego Mello da Silva

Formiga-MG
17 de abril de 2026

Sumário

1	Introdução	1
2	Parte II - Modelagem	1
2.1	Entrega 01 - Modelagem e Validação	1
2.2	Entrega 02 - Apresentação	2
3	Parte III	3
3.1	Entrega 03 - Protótipo de <i>Software</i> de Decisão	3
4	Barema	4
5	Considerações Finais	4
A	Pacotes de Programação Linear em Python 3	5
B	Exemplo Biblioteca PuLP	6
C	Exemplo Biblioteca Pyomo	7
D	Exemplo Biblioteca OR-Tools	8

1 Introdução

Este documento apresenta a especificação da 2a. e 3.a parte **Trabalho Prático** da disciplina Pesquisa Operacional, no valor de 20 pontos. O trabalho pode ser feito em grupo de até 03 (três) pessoas, e cada grupo deverá escolher obrigatoriamente um tema diferente para trabalhar.

Na etapa anterior, o trabalho consistiu em identificar um problema real de empresas do município de Formiga e região que pode ser resolvido por meio de programação linear. Nas etapas seguintes, o objetivo é propor e implementar um modelo de decisão que represente o problema por uma função objetivo linear, restrições lineares de igualdade e desigualdade, e variáveis de decisão contínuas.

O modelo será implementado computacionalmente e resolvido pelo método Simplex utilizando pacotes disponíveis em **Python**. O trabalho divide-se em duas partes: a primeira envolve a formulação de um modelo de programação linear alinhado às necessidades da empresa; a segunda compreende a implementação e resolução do modelo em **Python**, integrando-o a um protótipo de *software* para apoio à decisão.

As entregas poderão ser avaliadas de maneira *ad hoc* pelo professor da disciplina, com ou sem avaliadores convidados. Se julgar necessário, o professor poderá optar por arguir um grupo. Apresentações parciais também poderão ser requeridas, se houver necessidade.

Em todo o processo o grupo deverá fazer o papel de analista de pesquisa operacional, desde a identificação do empreendimento alvo (familiar, público, privado, etc), na coleta de informações, validação do modelo e implementação. Por questões didáticas cada uma das partes será apresentada a seguir, com detalhes sobre suas entregas parciais.

2 Parte II - Modelagem

A primeira parte do trabalho consiste na identificação de uma oportunidade de otimização de processo em empresa, instituição ou empreendimento da cidade de Formiga/MG ou região. Nesta seção estão descritas as entregas parciais que compreendem esta fase do trabalho, e formatos de documentos esperados.

2.1 Entrega 01 - Modelagem e Validação

Nesta entrega o objetivo é construir, encontrar ou adaptar um modelo de programação linear pronto para resolver o problema da empresa, instituição ou empreendimento familiar escolhida para este trabalho. Sugere-se a leitura de livros, artigos e monografias/dissertações/teses para seleção do modelo mais apropriado para a oportunidade de otimização identificada. Não há exigência que o modelo seja muito

complexo, mas sim que ele ajude a empresa a tomar melhores decisões.

O modelo deverá ser apresentado no formato AMPL, com destaque para a função objetivo, restrições e recursos disponíveis. Deve ser acompanhado de texto explicativo sobre o porquê de cada restrição do modelo, e como cada limitação foi identificada na visita com a empresa.

2.2 Entrega 02 - Apresentação

Esta entrega tem por objetivo confeccionar um material de apresentação sobre todo o levantamento feito neste trabalho. Espera-se uma apresentação no formato .pdf com os dados obtidos da empresa, oportunidade escolhida e modelo de decisão escrito na forma de programa linear. Caso o modelo escolhido ou adaptado tenha sido selecionado da literatura especializada é preciso citar a fonte. A apresentação deverá conter os seguintes *slides*:

- Capa: identificação do grupo e empresa escolhida
- Sumário Executivo: descrição dos principais dados da empresa, tabelas que resumem informações, fotos, ilustrações sobre seus processos e tudo o que o grupo julgar necessário para demonstrar que aprofundou na prospecção da oportunidade.
- Problema investigado: descrição do problema de decisão identificado na empresa selecionada. Deve conter as opções de decisão possíveis, as restrições identificadas, disponibilidade de recursos e outros, todos descritos em linguagem natural. Deve-se informar o que se deseja otimizar (i.e., o que pretende minimizar ou maximizar).
- Modelo de PL: modelo de programação linear que será implementado em *software* usando a linguagem de descrição de modelos AMPL e o solver **glpsol**. O modelo deverá conter a função objetivo e restrições, descritas de maneira mais formal possível.
- Descrição do Modelo: para cada equação do *slide* anterior deverá haver um texto explicativo associado explicando a restrição em linguagem natural.

3 Parte III

3.1 Entrega 03 - Protótipo de *Software* de Decisão

O protótipo de *software* tem como objetivo apoiar a tomada de decisão em empresas de pequeno porte, utilizando um modelo matemático de otimização. Em especial, ele deverá implementar o modelo desenvolvido na Parte II deste trabalho prático. Espera-se que ele seja capaz de receber dados reais de um problema enfrentado pela empresa que tenha sido identificado pelo grupo e, a partir desses dados, gerar um modelo de programação linear ou inteira, que será resolvido automaticamente.

A entrada de dados não pedirá ao usuário para inserir diretamente os parâmetros do modelo matemático, como coeficientes de funções objetivo ou restrições. Em vez disso, o *software* solicitará os dados reais do problema, como a quantidade de recursos disponíveis, demandas de produção, custos, tempos de fabricação, entre outros. Estes dados serão, então, convertidos internamente em coeficientes para compor a função objetivo e as restrições do modelo de otimização.

Para exemplificar, seja um problema de produção identificado. Usando o protótipo de *software*, o empresário poderá informar o número de produtos a serem fabricados, os recursos disponíveis, os custos de produção e os prazos de entrega, por exemplo. O *software*, com base nesses dados, gerará a função objetivo (como minimizar custos ou maximizar lucros) e as restrições (como capacidade de produção e demanda) automaticamente, convertendo os dados fornecidos nas variáveis apropriadas para o modelo matemático. Em alguns casos, os dados informados poderão ser agregados, sumarizados, e transformados até virar um coeficiente determinístico tal como requerido pelos modelos de programação matemática.

O modelo será, então, resolvido utilizando algum pacote de otimização em **python 3** (ver Apêndice A para detalhes sobre pacotes disponíveis). A solução será então apresentada ao usuário, que receberá um relatório com os resultados, incluindo os valores das variáveis de decisão e o valor otimizado da função objetivo em formato que o empresário consiga compreender, ou seja, não-técnico e na terminologia que é de seu costume. Informações como a quantidade ideal de cada item a ser produzido, os custos totais ou outras informações relevantes para a tomada de decisão devem estar presentes neste relatório. Esse relatório poderá ser exportado em formatos como texto simples, **.pdf**, **.html** ou formato de planilha **.csv**. Caso o grupo opte por formatos deste tipo, e não texto plano, é de responsabilidade procurar pacotes na linguagem capaz de fazer essa conversão - assim como informar instruções e instalação para avaliação do trabalho prático no computador do professor.

Como último requisito, o *software* será uma ferramenta *stand-alone*, ou seja, não Web. Ele poderá ser executado em sistemas operacionais Windows e Linux, com uma interface simples, que pode ser tanto gráfica (GUI), via arquivo quanto por linha de comando (CLI), ou ainda via arquivo do tipo **.csv** dependendo da escolha. A solução será modular, permitindo melhorias e adições de funcionalidades no futuro, conforme as necessidades da empresa se alterarem. Pensem neste protótipo como

uma primeira tentativa de estabelecer um projeto maior com a empresa, que poderá evoluir e tornar-se um produto de *software* capaz de resolver problemas em empresas similares de Formiga e região. Para quem desenvolve em Linux, sugere-se a biblioteca TKinter.

4 Barema

O barema de correção deste trabalho prático será feito mediante os critérios:

Critério	Pontos
Entrega 01 - Modelagem e Validação	10.0
Entrega 02 - Apresentação	5.0
Entrega 03 - Protótipo de <i>Software</i> de Decisão	15.0
TOTAL	30.0

5 Considerações Finais

Este trabalho tem por objetivo envolver o grupo nas fases de um projeto de pesquisa operacional, culminando na construção de uma aplicação customizada para um problema de decisão de empresa, instituição ou empreendimento familiar específico. Para tal é mandatório que cada grupo escolha seu próprio problema de decisão, não sendo aceitas duplicatas. A escolha da linguagem, a proposição e validação da formulação de programação linear/inteira e a modelagem do *software*, assim como a pesquisa nos guias de referência de bibliotecas que implementam o Simplex/ *Branch-and-Cut*, e outros problemas de ordem técnica são de responsabilidade do grupo.

No trabalho prático, a primeira parte foi dedicada ao levantamento de dados e identificação de problemas enfrentados por empresas de pequeno porte de Formiga/MG e região. A segunda parte envolveu a construção e validação do modelo de programação matemática para otimização. Agora, na terceira parte, o protótipo de software será desenvolvido para automatizar o processo de tomada de decisão, fornecendo uma ferramenta prática e eficiente para as empresas resolverem seus problemas de forma otimizada.

Quando concluído, permitirá aos membros do grupo (i) ter contato com uma situação real que requer o levantamento de oportunidades, requisitos e priorização de demandas em projetos de desenvolvimento de *software*; (ii) aplicação do conhecimento obtido na disciplina para resolver situações práticas; (iii) prototipação de soluções para validação em ambiente controlado.

A Pacotes de Programação Linear em Python 3

Nesta seção iremos apresentar brevemente alguns pacotes disponíveis em `python` que permitem implementar modelos de programação linear (LP), programação inteira (IP), e programação inteira mista (MIP), programação não-linear (NLP) e combinatória. Nas próximas seções são dados exemplos de como se pode implementar um modelo simples, didático, cuja formulação é dada a seguir. Trata-se do modelo *Produzir ou Comprar* apresentado nas aulas de modelagem, para decidir a produção ou compra de três modelos de motores:

$$\begin{aligned}
 \min \quad & 50x_1 + 90x_2 + 120x_3 + 65x_4 + 92x_5 + 140x_6 && \text{(Custo Total)} \\
 \text{s.t.} \quad & 1x_1 + 2x_2 + 0.5x_3 \leq 6000 && \text{(Montagem)} \\
 & 2.5x_1 + 1x_2 + 4x_3 \leq 10000 && \text{(Acabamento)} \\
 & x_1 + x_4 = 3000 && \text{(Dem. Motor 1)} \\
 & x_2 + x_5 = 2500 && \text{(Dem. Motor 2)} \\
 & x_3 + x_6 = 500 && \text{(Dem. Motor 3)} \\
 & x_1, x_2, x_3, x_4, x_5, x_6 \geq 0 && \text{(Não-Negatividade)}
 \end{aligned}$$

As opções de pacotes para LP, MILP e NLP em `python` mais populares são dadas a seguir, na Tabela 2. Todas são escolhas adequadas para implementar o módulo que faz papel de “motor” de decisão do protótipo. Observe que em alguns casos é preciso fazer a instalação prévia de algum *solver* comercial, como o Gurobi ou CPLEX, ou de código aberto, como o GLPK ou o CBC¹. O CBC é, inclusive, o *solver* padrão de alguns destes pacotes.

Tabela 2: Comparação entre bibliotecas PuLP, Pyomo e OR-Tools

	PuLP	Pyomo	OR-Tools
Descrição	Biblioteca simples para modelagem de LP e MIP.	Framework flexível para otimização avançada.	Biblioteca para otimização combinatória e MIP.
Origem	Comunidade <i>Open Source</i>	Sandia National Labs	Google
Resolve	LP, MIP	LP, MIP, NLP	LP, MIP, Combinatória
Padrão	CBC	Nenhum	CBC
Suporta	CBC, GLPK, Gurobi, CPLEX	GLPK, CBC, IPOPT, Gurobi, CPLEX	CBC, SCIP, Gurobi, CPLEX
<i>Solver</i>	CBC já incluído	<code>apt-get install glpk</code>	CBC já incluído
Pacote	<code>pip3 install pulp</code>	<code>pip3 install pyomo</code>	<code>pip3 install ortools</code>
Facilidade	Simples	Moderada	Intermediária

¹O CBC (*Coin-or Branch and Cut*) é um solver de código aberto para resolver problemas de Programação Linear Inteira Mista (*MILP - Mixed Integer Linear Programming*). Ele faz parte do projeto COIN-OR (*COmputational INfrastructure for Operations Research*) e é amplamente utilizado para resolver problemas de otimização de maneira eficiente.

B Exemplo Biblioteca PuLP

```
from pulp import LpProblem, LpVariable, LpMinimize, lpSum, value, PULP_CBC_CMD

# Criar o problema de otimização
problema = LpProblem("Produzir_ou_Comprar", LpMinimize)

# Variáveis de decisão
x1 = LpVariable("x1", lowBound=0, cat="Continuous") # Motores tipo 1 fabr.
x2 = LpVariable("x2", lowBound=0, cat="Continuous") # Motores tipo 2 fabr.
x3 = LpVariable("x3", lowBound=0, cat="Continuous") # Motores tipo 3 fabr.
x4 = LpVariable("x4", lowBound=0, cat="Continuous") # Motores tipo 1 terc.
x5 = LpVariable("x5", lowBound=0, cat="Continuous") # Motores tipo 2 terc.
x6 = LpVariable("x6", lowBound=0, cat="Continuous") # Motores tipo 3 terc.

# Função objetivo
problema += lpSum([50*x1, 90*x2, 120*x3, 65*x4, 92*x5, 140*x6]), "Custo Total"

# Restrições
problema += x1 + 2 * x2 + 0.5 * x3 <= 6000, "Restrição de Montagem"
problema += 2.5 * x1 + x2 + 4 * x3 <= 10000, "Restrição de Acabamento"
problema += x1 + x4 == 3000, "Demanda Motor 1"
problema += x2 + x5 == 2500, "Demanda Motor 2"
problema += x3 + x6 == 500, "Demanda Motor 3"

# Resolver o problema com CBC
problema.solve(PULP_CBC_CMD())

# Obter os resultados com CBC
resultado_cbc = {
    "Status": problema.status,
    "Custo Total": value(problema.objective),
    "Variáveis": {v.name: v.varValue for v in problema.variables()}
}

# Resultados
resultado_cbc
```


C Exemplo Biblioteca Pyomo

```
from pyomo.environ import ConcreteModel
from pyomo.environ import Var
from pyomo.environ import Objective
from pyomo.environ import Constraint
from pyomo.environ import SolverFactory
from pyomo.environ import NonNegativeReals

# Criar o modelo
m = ConcreteModel()

# Variáveis de decisão
m.x1 = Var(domain=NonNegativeReals) # Motores tipo 1 fabr.
m.x2 = Var(domain=NonNegativeReals) # Motores tipo 2 fabr.
m.x3 = Var(domain=NonNegativeReals) # Motores tipo 3 fabr.
m.x4 = Var(domain=NonNegativeReals) # Motores tipo 1 terc.
m.x5 = Var(domain=NonNegativeReals) # Motores tipo 2 terc.
m.x6 = Var(domain=NonNegativeReals) # Motores tipo 3 terc.

# Função objetivo
m.obj = Objective(
    expr = 50 * m.x1 + 90 * m.x2 + 120 * m.x3 +
          65 * m.x4 + 92 * m.x5 + 140 * m.x6,
    sense=1 # Minimizar
)

# Restrições
m.montagem = Constraint(expr=m.x1 + 2 * m.x2 + 0.5 * m.x3 <= 6000)
m.acabamento = Constraint(expr=2.5 * m.x1 + m.x2 + 4 * m.x3 <= 10000)
m.demanda1 = Constraint(expr=m.x1 + m.x4 == 3000)
m.demanda2 = Constraint(expr=m.x2 + m.x5 == 2500)
m.demanda3 = Constraint(expr=m.x3 + m.x6 == 500)

# Resolver o modelo
solver = SolverFactory('glpk') # Precisa instalar GLPK antes
results = solver.solve(m, tee=False)

# Obter os resultados
resultado_pyomo = {
    "Status": results.solver.status,
    "Custo Total": m.obj(),
    "Variáveis": {
        "x1": m.x1.value,
        "x2": m.x2.value,
        "x3": m.x3.value,
        "x4": m.x4.value,
        "x5": m.x5.value,
        "x6": m.x6.value,
    }
}
resultado_pyomo
```

D Exemplo Biblioteca OR-Tools

```
from ortools.linear_solver import pywraplp

# Criar o solver, selecionando o CBC
solver = pywraplp.Solver.CreateSolver('CBC')

# Variáveis de decisão
x1 = solver.NumVar(0, solver.infinity(), 'x1') # Motores tipo 1 fabricados
x2 = solver.NumVar(0, solver.infinity(), 'x2') # Motores tipo 2 fabricados
x3 = solver.NumVar(0, solver.infinity(), 'x3') # Motores tipo 3 fabricados
x4 = solver.NumVar(0, solver.infinity(), 'x4') # Motores tipo 1 terceirizados
x5 = solver.NumVar(0, solver.infinity(), 'x5') # Motores tipo 2 terceirizados
x6 = solver.NumVar(0, solver.infinity(), 'x6') # Motores tipo 3 terceirizados

# Função objetivo
objective = solver.Objective()
objective.SetCoefficient(x1, 50)
objective.SetCoefficient(x2, 90)
objective.SetCoefficient(x3, 120)
objective.SetCoefficient(x4, 65)
objective.SetCoefficient(x5, 92)
objective.SetCoefficient(x6, 140)
objective.SetMinimization()

# Restrições
solver.Add(x1 + 2 * x2 + 0.5 * x3 <= 6000) # Restrição de Montagem
solver.Add(2.5 * x1 + x2 + 4 * x3 <= 10000) # Restrição de Acabamento
solver.Add(x1 + x4 == 3000) # Demanda Motor 1
solver.Add(x2 + x5 == 2500) # Demanda Motor 2
solver.Add(x3 + x6 == 500) # Demanda Motor 3

# Resolver o problema
status = solver.Solve()

status_map = {
    pywraplp.Solver.OPTIMAL: "OPTIMAL",
    pywraplp.Solver.FEASIBLE: "FEASIBLE",
    pywraplp.Solver.INFEASIBLE: "INFEASIBLE",
    pywraplp.Solver.UNBOUNDED: "UNBOUNDED",
    pywraplp.Solver.ABNORMAL: "ABNORMAL",
    pywraplp.Solver.NOT_SOLVED: "NOT_SOLVED"
}

# Coletar resultados
if status == pywraplp.Solver.OPTIMAL:
    resultado_ortools = {
        "Status": status_map[status],
        "Custo Total": solver.Objective().Value(),
        "Variáveis": {
            "x1": x1.solution_value(),
            "x2": x2.solution_value(),
            "x3": x3.solution_value(),
            "x4": x4.solution_value(),
            "x5": x5.solution_value(),
            "x6": x6.solution_value(),
        }
    }
else:
    resultado_ortools = {"Status": "Não foi encontrada solução ótima."}

resultado_ortools
```